StudyFlow

Project Portfolio

Computer & Media Lab

Project Manager:	Karthik Polpakkara Mana
Front-end Developer:	Chirag Nagesh
Quality & Presentation:	Anish Sharma
Technology:	Python, Flask, SQL, HTML & CSS

Contents

1 Problem Statement, Research and Pitch **2**
- 1.1 The Problem . . . 2
- 1.2 Our Pitch (The MVP Goal) . . . 2
- 1.3 Market Differentiation . . . 2

2 Project Plan, Risks and Progress **2**
- 2.1 The Original Roadmap . . . 2
- 2.2 RAID Log (Risks, Assumptions, Issues, Dependencies) . . . 2
- 2.3 The Big Pivot (Progress Report) . . . 2

3 User Journeys and Design **3**
- 3.1 User Journey . . . 3
- 3.2 Design & Media Sources . . . 3

4 Product Version and Setup Instructions **3**
- 4.1 Tech Stack Architecture . . . 3
- 4.2 Setup Instructions (Local Development) . . . 3
- 4.3 User Guide . . . 3

5 Test Results, Limitations and Reflections **4**
- 5.1 What We Built (Tested Features) . . . 4
- 5.2 Limitations & Next Steps . . . 4
- 5.3 Individual Reflections . . . 4

1 Problem Statement, Research and Pitch

1.1 The Problem

Busy university students need a way to assemble a personalised weekly and daily plan in a few minutes because existing planner apps are cluttered with unused features, require long setup, and force users into a default look that does not match their taste. As a result, students abandon these tools and fall back on scattered notes.

1.2 Our Pitch (The MVP Goal)

StudyFlow is an anti-bloat, fast, and simple planner. The MVP goal is to prove that a single student can build a week's plan, add a task with a priority, and receive a mock reminder, with all data saved locally. We promised a timetable setup time of 5 to 10 minutes.

1.3 Market Differentiation

Unlike feature-heavy apps (e.g., MyStudyLife) that add AI, complex cloud syncing, and grade trackers, or template-based tools (e.g., Notion) that could take 30+ minutes to set up, StudyFlow is strictly anti-bloat. We prioritize speed, intentional constraints, and free aesthetic personalization.

2 Project Plan, Risks and Progress

2.1 The Original Roadmap

Our initial project plan was structured in three phases:

1. **Terminal prototype:** Test the core workflow in a simple command-line version.
2. **Windows desktop app:** Grow the prototype into a full desktop application.
3. **Linux support:** Explore later if time and resources allowed.

The desktop plan is why we first picked **Rust** as our programming language.

2.2 RAID Log (Risks, Assumptions, Issues, Dependencies)

ID	Risk / Issue	Status / Mitigation
R1	Scope creep (adding AI or heavy features)	**Mitigated:** Strict anti-bloat stance.
R2	Tooling failure (Rust IDE crashing)	**Realised:** Pivot to Python/Flask.
R3	Limited team experience in Rust	**Monitored:** Only one member could code.

2.3 The Big Pivot (Progress Report)

The Issue: During the development of the Rust prototype, our IDE kept crashing and could not load Rust at all. Development stalled completely. Furthermore, only one team member had the Rust skills to contribute.

The Decision: We made the agile decision to pivot our tech stack to **Python, Flask, SQL, and HTML/CSS**. This aligned with the team's existing skills and unblocked the project. This pivot is a key example of project risk management in action.

3 User Journeys and Design

3.1 User Journey

The core user journey is intentionally frictionless:

1. **Entry:** The student opens the StudyFlow web app (via a terminal).
2. **Action:** They use the Quick-Add form to enter a task or a weekly timetable block.
3. **Processing:** Flask processes the request and saves the data to the SQL database.
4. **Output:** The task appears instantly in the Daily View or Weekly Timetable.

3.2 Design & Media Sources

To fulfill the aesthetic requirement, we implemented a CSS-based theme system (e.g., "Cozy" and "Minimal" themes). Users can toggle themes, which changes the color palette and border radius without requiring any manual setup. We used standard web typography (sans-serif) to keep the UI clean and accessible.

4 Product Version and Setup Instructions

4.1 Tech Stack Architecture

- **HTML & CSS (Front-end):** Built by Chirag. Provides the visual forms, daily views, and weekly grids.
- **Flask (Web Framework):** Built by Karthik. Lightweight, routing requests from the web pages to the Python functions.
- **SQL (Database):** Stores every task and event locally so data persists when the app closes.

4.2 Setup Instructions (Local Development)

To run StudyFlow on a local machine (tested on Linux/CachyOS):

1. Clone the project repository and navigate to the folder.
2. Create and activate a virtual environment:
   ```
   python -m venv venv
   source venv/bin/activate
   ```
3. Install the Flask dependency:
   ```
   pip install -r requirements.txt
   ```
4. Initialize and run the app:
   ```
   python app.py
   ```
5. Open a web browser and navigate to `http://127.0.0.1:5000`

Note: Using VS Code to run this project on a Windows PC, will let you skip initiating a virtual environment and let's you immediately 'pip install -r requirements.txt'.

4.3 User Guide

- **Adding a Task:** Use the "New Task" form on the Week View. Provide a title, priority (High/Medium/Low), and estimated duration.

- **Adding a Timetable Block:** Use the "New Timetable Block" form to assign a class or study session to a specific day and time.
- **Viewing Progress:** Navigate to the "Today View". Check off tasks as they are completed and watch the progress bar update.
- **Switching Themes:** Click the "Switch Theme" button in the navigation bar.

5 Test Results, Limitations and Reflections

5.1 What We Built (Tested Features)

We successfully built and tested the following simple functions:

- **Add tasks:** Enter a new task or event into the planner.
- **View daily events:** See everything planned for today in one place.
- **View weekly events:** See the whole week at a glance.
- **Save and load data:** Keep tasks in the SQL database between sessions.

5.2 Limitations & Next Steps

- **Reminders:** We have not yet implemented actual push notifications. The next step is to finish the MVP's mock reminder notification system.
- **Progress Tracking:** We need to refine the daily progress calculation and add historical data.
- **User Testing:** We still need to conduct external usability testing with students outside our team to validate the 5-10 minute setup claim.

5.3 Individual Reflections

Karthik Polpakkara Mana (Project Manager & Backend)

Reflection: Coming up with the StudyFlow idea was easy; managing the pivot was the hard part. When the Rust IDE crashed, I had to make a fast decision to abandon our original tech stack to save the project. I learned that choosing tools that fit the team's skills is more important than choosing the "ideal" technology. Writing the Python, Flask, and SQL code myself allowed me to unblock the team and play to their strengths. My next focus is ensuring we hit our testing milestones.

Chirag Nagesh (Front-end Developer)

Reflection: My HTML and CSS skills ended up shaping the entire technical direction of the project. When we switched to a web app, I was able to quickly build out the forms and the weekly timetable grid. It was rewarding to see the SQL backend connect seamlessly with the frontend I designed.

Anish Sharma (Quality & Presentation)

Reflection: My role was to review our ideas and judge whether they were good enough for the scope of the course, while leading the presentation work. I helped enforce the "anti-bloat" rule, ensuring we didn't add features we couldn't finish. Checking the quality of what we produced taught me that small, simple functions make code much easier to test and present.